

Adapter design pattern::

Adapter Design Pattern – Application Flow Explanation

Our application follows an **adapted Adapter-based design pattern** to bridge HTTP requests with the service layer and enforce a structured business pipeline.

Why we use this pattern

- To **decouple controllers from business logic**
 - To enforce a **standard execution pipeline** for every request
 - To centralize **authorization, DB handling, and post-processing**
 - To make the system **extensible and maintainable**
-

Request Flow (End-to-End)

1. Controller Layer

- Receives the HTTP request
- Performs minimal request validation
- Creates an instance of the **Adapter**

2. Adapter Layer

- Acts as a bridge between the HTTP layer and business logic
- Translates request data into a business-friendly format
- Passes control to the **Execution Service**
- Ensures the controller remains thin

3. Execution Service (Business Pipeline)

- Executes a standardized pipeline:
 - User authentication & role validation

- Permission checks
- Connection handling
- Prepared statement creation
- Database operations
- Post-operation validations (audit logs, response checks, cleanup)

4. **Specific Business Service**

- Contains actual business logic
- Executes domain-specific rules
- Interacts with the database layer

5. **Database Layer**

- Performs CRUD operations using prepared statements
- Returns results to the service layer

6. **Post-Processing**

- Result validation
- Error handling
- Audit/logging (if applicable)
- Response preparation

7. **Response Flow**

- Travels back through Execution Service → Adapter → Controller
 - Controller sends the final HTTP response
-

High-Level Flow Summary

Controller



Adapter



Execution Service (Business Pipeline)



Specific Service



Database



Post-Operation Checks



Response back to Controller

Key Benefits

- **Loose coupling** between layers
 - **Single execution pipeline** for all business flows
 - **Easy to add new services** without touching controllers
 - **Cleaner controllers, reusable business logic**
 - **Consistent security and DB handling**
-
-

Example: Create Audit Assignment Flow

1 Controller (Thin – no business logic)

```
@RestController
@RequestMapping("/assignments")
public class AssignmentController {

    private final AssignmentAdapter assignmentAdapter;

    public AssignmentController(AssignmentAdapter assignmentAdapter)
    {
        this.assignmentAdapter = assignmentAdapter;
    }

    @PostMapping
    public ResponseEntity<?> createAssignment(@RequestBody
AssignmentRequest request) {
        return assignmentAdapter.createAssignment(request);
    }
}
```

👉 Controller only:

- Accepts HTTP request
- Delegates to Adapter
- No DB, no security, no logic

2 Adapter (HTTP → Business Bridge)

```
@Component
public class AssignmentAdapter {

    private final ExecutionService executionService;

    public AssignmentAdapter(ExecutionService executionService) {
        this.executionService = executionService;
    }
}
```

```

    public ResponseEntity<?> createAssignment(AssignmentRequest
request) {

        AssignmentContext context = new AssignmentContext();
        context.setRequest(request);

        ExecutionResult result = executionService.execute(
            context,
            BusinessOperation.CREATE_ASSIGNMENT
        );

        return ResponseEntity.status(result.getStatus())
            .body(result.getResponse());
    }
}

```

👉 Adapter responsibilities:

- Converts HTTP request to **business context**
- Calls execution pipeline
- Converts business response → HTTP response

3 Execution Service (Business Pipeline – Core)

This is where your **adapted design pattern** really lives.

```

@Service
public class ExecutionService {

    private final AuthService authService;
    private final BusinessServiceFactory serviceFactory;

    public ExecutionResult execute(BusinessContext context,
                                   BusinessOperation operation) {

        // 1. Authentication & Role validation
        authService.validateUser(context.getUser(), operation);
    }
}

```

```

        // 2. Get business service
        BusinessService service =
            serviceFactory.getService(operation);

        // 3. Execute business logic
        Object result = service.execute(context);

        // 4. Post-operation checks
        audit(context, result);

        return ExecutionResult.success(result);
    }

    private void audit(BusinessContext context, Object result) {
        // logging / audit trail / validation
    }
}

```

👉 Pipeline enforced here:

- Security
- Service routing
- Execution
- Post-processing

4 Business Service Interface (Template-like)

```

public interface BusinessService {
    Object execute(BusinessContext context);
}

```

5 Specific Business Service (Actual Logic)

```

@Service
public class CreateAssignmentService implements BusinessService {

    private final AssignmentRepository repository;
}

```

```

    public CreateAssignmentService(AssignmentRepository repository)
    {
        this.repository = repository;
    }

    @Override
    public Object execute(BusinessContext context) {

        AssignmentRequest req =
            ((AssignmentContext) context).getRequest();

        // Prepared statement via JPA / JDBC
        Assignment assignment = new Assignment();
        assignment.setName(req.getName());
        assignment.setCreatedBy(context.getUser().getId());

        return repository.save(assignment);
    }
}

```

👉 Only **business logic**, nothing else.

6 Factory (Routes Operation → Service)

```

@Component
public class BusinessServiceFactory {

    private final Map<BusinessOperation, BusinessService> services;

    public BusinessServiceFactory(List<BusinessService> serviceList)
    {
        services = Map.of(
            BusinessOperation.CREATE_ASSIGNMENT,
            serviceList.stream()
                .filter(s -> s instanceof CreateAssignmentService)
                .findFirst()
                .orElseThrow()
        );
    }
}

```

```
    public BusinessService getService(BusinessOperation operation) {  
        return services.get(operation);  
    }  
}
```

7 Flow Recap (Interview-friendly)

HTTP Request

↓

Controller (thin)

↓

Adapter (HTTP → Business)

↓

Execution Service (pipeline)

↓

Specific Business Service

↓

Database

↑

Post-operation checks

↑

HTTP Response

Patterns Used (Tell it straight)

- **Adapter Pattern** – Controller → Adapter → Service
- **Template Method (adapted)** – Fixed execution pipeline
- **Factory Pattern** – Resolve business service at runtime
- **Single Responsibility** – Each layer does one thing

Roles and responsibility

Roles and responsibility

=====

- Actively involved in development activities, with approximately 80% focus on new feature development and enhancements.
- Handle around 20% of sprint capacity for production support, client-reported defects, and bug fixes, ensuring minimal impact on business operations.
- Participate in technical and architectural discussions, contributing to design decisions, scalability considerations, and best practices.
- Actively participate in Agile Scrum ceremonies including:
 - Daily stand-ups
 - Sprint planning
 - Backlog grooming
 - PI increment planning
- Perform peer code reviews to ensure code quality, adherence to coding standards, performance optimization, and maintainability.

SDLC

Software Development Life Cycle (SDLC)

1. Feature discussion and high-level planning are carried out by higher management and product leadership.
2. Approved features are broken down into user stories (tickets) by the entire team, including developers and QA.
3. Steps 1 and 2 are part of PI (Program Increment) Planning, which is done for the upcoming 3 months of development and QA work.
4. Before the start of every sprint, stories are groomed, refined, estimated, and assigned to respective developers and QA engineers.
5. During sprint planning, stories are assigned to developers. Based on the number of stories (for example, 10 tickets) and available QA resources (for example, 2 QA), QA planning is done for functional and feature-wise testing.
6. Story estimation is done using story points, where:
 - 1 story point represents approximately 1 day (8 hours) of work
 - A story typically contains 2 to 3 story points
7. Once development for a story is completed, it is handed over to the QA team for functional and regression testing.
8. If QA identifies any issues, they are reported as internal bugs.
9. If a bug is related to a story developed by a specific developer, the bug is assigned back to the same developer for fixing.
10. After bug fixes, the QA team re-tests the functionality to ensure the issue is resolved and no regressions are introduced.
11. Once QA sign-off is completed, the feature is tested in the UAT environment by the Product Manager or business stakeholders.
12. As the PI approaches completion, a code freeze is applied on the staging branch.
13. After code freeze: (checked once)
 - Automation testers and developers run automated test suites
 - Developers execute unit and integration test cases
 - DevOps runs integration pipelines
 - Performance team conducts performance and load testing
14. Development activities continue on the main/UAT branch while the staging branch is used for testing and sanity validation.
15. After all validations are successful, the final build is released to production.

SDLC Flow for Client-Reported Bug

1. A bug is reported by the client and logged as a production issue or client defect in the tracking system.
2. The issue is analyzed by the team for impact, severity, and priority.
3. Based on the analysis, a dedicated team or developer is assigned to handle the bug.
4. The bug is planned and included in the next sprint under the allocated support capacity (typically 20% of the sprint).
5. The assigned developer works on the fix, following standard development practices including code changes and unit testing.
6. Once the fix is completed, the bug is moved to QA for validation and regression testing.
7. If QA finds any issues, the bug is reassigned to the developer for correction.
8. After successful QA validation, the fix is deployed to UAT for verification by the product manager or client (if required).
9. Once approved, the fix is merged into the staging branch and included in the next planned release.
10. After release, the fix is monitored in production to ensure the issue is fully resolved and no new issues are introduced.

Feature Deployment

RELEASE MANAGEMENT & DEPLOYMENT FLOW

1. FEATURE DEVELOPMENT PHASE

- Developers work on individual feature branches created from the main development branch.
- Each feature follows:
 - Code implementation
 - Unit testing
 - Local verification
 - Peer code reviews (PRs)
- All merges are done only after review approval and static checks.

2. CODE FREEZE

- Once all planned features for the release are completed, a code freeze is declared.
- During code freeze:
 - No new features are allowed.
 - Only critical bug fixes, security fixes, or release blockers are permitted.
 - Any fix requires explicit approval.
- This ensures stability and prevents scope creep before release.

3. STAGING BRANCH CREATION

- After code freeze, a staging branch is created from the stabilized development branch.
- The staging branch represents a production-like snapshot of the application.
- No direct feature development happens on this branch.

4. STAGING ENVIRONMENT DEPLOYMENT

- The staging branch is deployed to the staging environment.
- This environment closely mirrors production in terms of:
 - Infrastructure
 - Configuration
 - Data structure
 - Security settings

5. AUTOMATION & TEST EXECUTION

- Once deployed to staging, the CI/CD pipeline is triggered.
- The following automated tests are executed:
 - Unit tests
 - Integration tests
 - API contract tests
 - Regression test suites
 - Security and vulnerability scans (if applicable)
- Any test failure blocks the release and sends feedback to the team.

6. MANUAL & BUSINESS VALIDATION

- After automation passes, manual testing is performed:
 - Functional testing
 - Exploratory testing
 - User acceptance testing (UAT)

- Business stakeholders validate:
 - Feature correctness
 - Data integrity
 - End-to-end workflows

7. RELEASE BRANCH CREATION⁴⁵

- Once staging is fully validated, a release branch is created from the staging branch.
- This branch is strictly controlled and used only for:
 - Final bug fixes
 - Versioning
 - Release documentation
- No new features are allowed at this stage.

8. PRODUCTION DEPLOYMENT

- The release branch is deployed to the production environment via the CI/CD pipeline.
- Deployment strategies may include:
 - Blue-green deployment
 - Rolling deployment
 - Canary releases
- Health checks and smoke tests are executed post-deployment.

9. POST-DEPLOYMENT VERIFICATION

- After production deployment:
 - Application logs are monitored
 - Metrics and alerts are observed
 - Performance and error rates are validated
- Immediate rollback plans are kept ready in case of issues.

10. RELEASE CLOSURE

- Once production stability is confirmed:
 - The release branch is tagged
 - Changes are merged back to the main branch
 - Release notes are finalized
- The system is now ready for the next development cycle.

KEY BENEFITS OF THIS PROCESS:

- Controlled and predictable releases
- Early detection of defects
- Production stability
- Clear separation of development, validation, and release stages
- Reduced risk during deployment

Bugs and interview que

Bugs faced in the project

1) Latency bug

=====

LATENCY BUG – TIME TRACKING MODULE (Spring Boot + JDBC)

=====

1) ISSUE + EXISTING CODE

ISSUE:

- Time Tracking screen was slow
- ~5000 records loaded in single request
- High latency and poor user experience

ROOT CAUSE:

- JDBC used with custom DTO mapper
- Entire dataset processed in application layer

EXISTING CODE:

DAO LAYER:

```
public List<TimeEntryDTO> getAllTimeEntries() throws SQLException {
```

```
    String query = "SELECT t.id, t.hours, u.name, t.date " +  
        "FROM time_entry t " +  
        "JOIN user u ON t.user_id = u.id";
```

```
    Connection conn = dataSource.getConnection();  
    PreparedStatement ps = conn.prepareStatement(query);  
    ResultSet rs = ps.executeQuery();
```

```
    List<TimeEntryDTO> result = new ArrayList<>();
```

```
    // PROBLEM: Loop runs ~5000 times
```

```
    while (rs.next()) {  
        TimeEntryDTO dto = new TimeEntryDTO();
```

```
        dto.setId(rs.getLong("id"));  
        dto.setHours(rs.getDouble("hours"));  
        dto.setUserName(rs.getString("name"));  
        dto.setDate(rs.getDate("date").toLocalDate());
```

```
        result.add(dto);
```

```
    }
```

```
    return result;
}
```

PROBLEMS:

- Loads full dataset into memory
- 5000 DTO objects created
- High CPU usage due to loop
- Large response payload
- Application layer bottleneck

2) FIXED CODE + PERFORMANCE IMPROVEMENT

FIX 1: PAGINATION (MAIN FIX)

```
public List<TimeEntryDTO> getTimeEntries(int offset, int limit) throws SQLException {
```

```
    String query = "SELECT t.id, t.hours, u.name, t.date " +
        "FROM time_entry t " +
        "JOIN user u ON t.user_id = u.id " +
        "ORDER BY t.id " +
        "LIMIT ? OFFSET ?";
```

```
    Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement(query);
```

```
    ps.setInt(1, limit);
    ps.setInt(2, offset);
```

```
    ResultSet rs = ps.executeQuery();
```

```
    List<TimeEntryDTO> result = new ArrayList<>();
```

```
    // Now loop runs only 50–100 times
    while (rs.next()) {
        result.add(mapRow(rs));
    }
```

```
    return result;
}
```

FIX 2: OPTIMIZED MAPPER

```
private TimeEntryDTO mapRow(ResultSet rs) throws SQLException {
```

```

return TimeEntryDTO.builder()
    .id(rs.getLong("id"))
    .hours(rs.getDouble("hours"))
    .userName(rs.getString("name"))
    .date(rs.getDate("date").toLocalDate())
    .build();
}

```

FIX 3: CONTROLLER

```

@GetMapping("/time-entries")
public List<TimeEntryDTO> getTimeEntries(
    @RequestParam int page,
    @RequestParam int size) throws SQLException {

    int offset = page * size;
    return service.getTimeEntries(offset, size);
}

```

PERFORMANCE IMPROVEMENT:

BEFORE:

- 5000 records processed
- 5000 DTO objects
- High CPU + memory usage
- Large payload → slow API

AFTER:

- Only 50–100 records per request
- Reduced loop iterations
- Lower object creation
- Smaller payload

RESULT:

- Faster response time
- Reduced CPU usage
- Better scalability

3) WHY JDBC INSTEAD OF JPA/HIBERNATE

REASON 1: FULL CONTROL OVER SQL

- We write exact queries
- No ORM-generated queries
- Better control over joins and execution

REASON 2: PERFORMANCE PREDICTABILITY

- No Hibernate overhead:
 - No persistence context
 - No dirty checking
- Direct DB interaction

REASON 3: AVOID ORM ISSUES

- No N+1 problem
- No lazy/eager confusion
- Easier debugging

REASON 4: LEGACY SYSTEM

- Existing system already using JDBC
- Migration to JPA is risky and costly

REASON 5: BETTER FOR LARGE DATA

- Can control:
 - Pagination (LIMIT/OFFSET)
 - Fetch size
- Avoid loading huge datasets into memory

ONE LINE INTERVIEW ANSWER:

We used JDBC instead of JPA to have better control over SQL, avoid ORM overhead, and handle large datasets efficiently in a performance-critical legacy system.

“If we used JPA, the manual JDBC code like Connection, PreparedStatement, and ResultSet would be replaced by Spring Data JPA repositories. Pagination would be handled using Pageable instead of LIMIT/OFFSET, and entity mapping would be automatic instead of manual loops. However, the core issue would still remain if we fetched all records at once, so pagination is the real fix, not just switching from JDBC to JPA.”

=====

2)BUG SCENARIO: UI FIELD NOT RENDERING – DATA COMPATIBILITY ISSUE

ISSUE:

Users reported that a specific text field was not visible on the UI while loading a particular screen. The field existed in the layout configuration but was either blank or not rendered properly in the new UI.

IMPACT:

- Users were unable to view or edit a critical test-related field.
- Issue was intermittent and data-specific, making it difficult to reproduce.

- Affected only a subset of existing records.

INITIAL INVESTIGATION:

- Attempted to reproduce the issue locally and in lower environments.
- Verified:
 - UI configuration
 - Field visibility rules
 - Role-based permissions
 - API response payload
- The issue could NOT be reproduced consistently.
- New records were working fine, indicating the problem was not in the UI logic itself.

HYPOTHESIS:

- After detailed discussion and comparison between affected and non-affected records, we suspected a data-related issue rather than a UI or backend logic issue.

DATA ANALYSIS:

- Identified the database column backing the problematic text field.
- Executed queries to extract all distinct values from that column for affected records.
- While analyzing the data, we found:
 - Presence of restricted / non-printable / legacy special characters
 - Characters introduced from an older version of the UI
 - These characters were not compatible with the new UI rendering engine

ROOT CAUSE:

- The old UI allowed certain unsupported characters to be persisted in the database.
- The new UI framework had stricter rendering and validation rules.
- As a result, the field failed to render when encountering those invalid characters.

SOLUTION IMPLEMENTED:

1. DATA CLEANUP SCRIPT

- Wrote a backend SQL/data conversion script to sanitize the affected column.
- Removed or replaced unsupported and non-renderable characters.
- Ensured data consistency without impacting valid content.

2. PREVENTIVE VALIDATION

- Added validation at the backend layer to prevent such characters from being saved in the future.
- Ensured backward compatibility with the new UI.

RESULT:

- The affected field started rendering correctly for all users.
- No UI changes were required.
- Issue resolved permanently across environments.

KEY LEARNINGS:

- Legacy data can break modern UI frameworks.
- Not all UI issues are UI bugs — data integrity plays a critical role.

- Proper data validation and cleanup are essential during UI migrations.

3) Us military client facing document upload issue ,filter was changed

- How were those bugs identified?

- How were they resolved?

1. What was your contribution? -> RBAC

2. What was your biggest challenge? How do you overcome that?

DUPLICATE SURVEY RESPONSE ISSUE (CONCURRENCY PROBLEM)

SYSTEM DESIGN OVERVIEW

We have separate tables and services:

Survey
Questions
Survey Assignment
Survey Response

Relationships:

Admin creates a Survey
A Survey contains multiple Questions
Same Question can be reused across surveys
A Survey can be assigned to multiple Users
A User can fill multiple Surveys

DATABASE STRUCTURE

Table: survey_response

Columns:

id (Primary Key, auto-generated)

user_id

survey_id

question_id

response

Business Rule:

Each (user_id, survey_id, question_id) combination must be unique.

PROBLEM FACED

UI was sending duplicate requests due to server slowness

Backend used "check-then-insert" logic

Method was not synchronized

Failure Flow:

Thread 1 → Check → No record found

Thread 2 → Check → No record found

Thread 1 → Insert

Thread 2 → Insert

Result:

Duplicate rows created in survey_response table

WHY APPLICATION-LEVEL SOLUTIONS WERE NOT USED

Synchronization:

Reduces performance

Blocks threads

Not suitable for high concurrency

Frontend Debouncing:

Not reliable

Cannot control retries or race conditions

Check-Then-Insert:

Not atomic

Leads to race conditions

FINAL SOLUTION

Moved consistency enforcement to database layer

Added composite UNIQUE constraint:

```
ALTER TABLE survey_response  
ADD CONSTRAINT uk_user_survey_question  
UNIQUE (user_id, survey_id, question_id);
```

HOW IT SOLVES THE ISSUE

Thread 1 → Insert → Success

Thread 2 → Insert → Fails (unique constraint violation)

Result:

Only one record is stored

Duplicate entries prevented

Data integrity maintained

APPLICATION-LEVEL HANDLING

```
try {  
    surveyResponseRepository.save(entity);  
} catch (DataIntegrityViolationException ex) {  
    log.warn("Duplicate response detected");  
}
```

HANDLING EXISTING DUPLICATES

Step 1: Find duplicates

```
SELECT survey_id, question_id, user_id, COUNT() AS cnt  
FROM survey_response  
GROUP BY survey_id, question_id, user_id  
HAVING COUNT() > 1;
```

Step 2: Verify duplicates

```
SELECT t1.*  
FROM survey_response t1  
JOIN survey_response t2  
ON t1.survey_id = t2.survey_id  
AND t1.question_id = t2.question_id  
AND t1.user_id = t2.user_id  
AND t1.id > t2.id;
```

Step 3: Delete duplicates (keep lowest id)

```
DELETE t1
FROM survey_response t1
JOIN survey_response t2
ON t1.survey_id = t2.survey_id
AND t1.question_id = t2.question_id
AND t1.user_id = t2.user_id
AND t1.id > t2.id;
```

Step 4: Add unique constraint

```
ALTER TABLE survey_response
ADD CONSTRAINT unique_survey_question_user
UNIQUE (survey_id, question_id, user_id);
```

NOTE:

Take backup before deletion
Constraint will fail if duplicates exist
Follow steps in order
KEY DESIGN INSIGHT

Do not rely on application logic for concurrency control.
Always enforce critical constraints at the database level.

FINAL OUTCOME
Duplicate entries removed
System handles concurrency safely
No performance impact
Data integrity ensured
Issue permanently resolved
INTERVIEW ANSWER (SHORT)

We faced duplicate entries due to concurrent requests and non-atomic check-then-insert logic. Instead of using synchronization, we added a composite unique constraint at the database level, which ensured data consistency without impacting performance.

4. How did you optimise your API?

=====

HOW I OPTIMIZED OUR APIs

=====

API optimization was not done in one place.
It was done at multiple layers:

1) Database layer

- 2) Caching layer
- 3) Application layer
- 4) Authorization layer
- 5) Infrastructure layer

1) DATABASE OPTIMIZATION

Problem:

- Slow queries in audit reports
- N+1 query issues
- Large joins across audit, risk, findings tables

What I did:

- ✓ Added proper indexing
 - Composite indexes on (audit_id, status)
 - Indexes on foreign keys
 - Indexes on frequently filtered columns
- ✓ Solved N+1 issue
 - Replaced lazy loops with JOIN FETCH / custom queries
 - Used DTO projections instead of full entity load
- ✓ Reduced payload size
 - Used pagination (Pageable)
 - Limited columns using projections
- ✓ Optimized reporting queries
 - Used native queries for heavy aggregation
 - Avoided unnecessary entity hydration

Impact:

- Query time reduced significantly
- Lower DB CPU usage
- Faster report APIs

2) REDIS CACHING OPTIMIZATION

Problem:

Authorization lookup hitting DB every request.

Solution:

- ✓ Implemented cache-aside pattern
- ✓ Stored user permissions in Redis SET